

Day 5：运维与部署实践 *□

*□今日时间安排（半天强化版）

时段	时间	内容	形式
上午	9:00-10:30	Docker Compose多服务编排	动手实践
	10:45-12:00	Nginx反向代理与负载均衡	配置实战
下午	14:00-15:30	CI/CD流水线搭建	自动化部署
	15:45-17:00	监控告警与日志收集	运维体系

✓✓ 学习目标

□今日完成后，你将能够：

✓ 编写Docker Compose - 编排多容器应用栈（Web+DB+Cache） ✓ 配置Nginx - 反向代理、SSL证书、静态资源服务 ✓ 搭建CI/CD - 自动化测试、构建、部署流程 ✓ 监控应用 - 日志收集、性能指标、健康检查 ✓ 生产部署 - 环境变量管理、安全加固、备份策略

** 详细学习内容

□1. Docker Compose 多服务编排（1.5小时）

□1.1 完整项目Docker化

项目结构：

```
blog-system/
+-- docker-compose.yml      # 编排文件
+-- docker-compose.prod.yml # 生产环境配置
+-- .env                    # 环境变量
+-- .env.example            # 环境变量示例
|
+-- backend/                # FastAPI后端
|   +-- Dockerfile
|   +-- requirements.txt
|   +-- main.py
|
+-- frontend/               # Vue3前端
|   +-- Dockerfile
|   +-- nginx.conf          # 前端Nginx配置
|
+-- nginx/                  # 主Nginx（反向代理）
|   +-- Dockerfile
|   +-- nginx.conf
|   +-- conf.d/
|       +-- backend.conf
|       +-- frontend.conf
|
+-- postgres/               # PostgreSQL初始化
|   +-- init.sql
|
+-- redis/                  # Redis配置
    +-- redis.conf
```

□ 1.2 docker-compose.yml 完整配置

```
version: '3.8'

services:
  # ===== 后端API服务 =====
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: blog-backend
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://${POSTGRES_USER*}:${POSTGRES_PASSWORD*}@db:5432/${POSTGRES_DB*}
      - REDIS_URL=redis://redis:6379/0
      - SECRET_KEY=${SECRET_KEY*}
      - ENVIRONMENT=${ENVIRONMENT:-development*}
    volumes:
      - ./backend:/app          # 开发时挂载源码，实现热重载
      - backend-data:/app/uploads # 用户上传文件持久化
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_healthy
    networks:
      - blog-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 40s
    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"

  # ===== 前端服务 =====
  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    args:
      - VITE_API_BASE_URL=${VITE_API_BASE_URL:-http://localhost:8000*}
    container_name: blog-frontend
    restart: unless-stopped
```

ports:

- "3000:80"

depends_on:

- backend

networks:

- blog-network

logging:

driver: json-file

options:

max-size: "10m"

max-file: "3"

===== Nginx反向代理 =====

nginx:

build:

context: ./nginx

dockerfile: Dockerfile

container_name: blog-nginx

restart: unless-stopped

ports:

- "80:80"

- "443:443"

volumes:

- `./nginx/nginx.conf:/etc/nginx/nginx.conf:ro`

- `./nginx/conf.d:/etc/nginx/conf.d:ro`

- certbot-etc:/etc/letsencrypt:ro

- certbot-var:/var/lib/letsencrypt:ro

- ./frontend/dist:/usr/share/nginx/html/frontend:ro

depends_on:

- backend

- frontend

networks:

- blog-network

healthcheck:

```
test: ["CMD", "curl", "-f", "http://localhost/health"]
```

interval: 30s

timeout: 5s

retries: 3

===== PostgreSQL数据库 =====

db:

image: postgres:15-alpine

container_name: blog-db

restart: unless-stopped

ports:

- "5432:5432"

environment:

POSTGRES_USER: \${POSTGRES_USER:-bloguser*}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD:?请设置数据库密码*}

POSTGRES_DB: \${POSTGRES_DB:-blogdb*}

PGDATA: /var/lib/postgresql/data/pgdata

volumes:

- postgres_data:/var/lib/postgresql/data

- ./postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

networks:

- blog-network

healthcheck:

```
test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-bloguser*} -d ${POSTGRES_DB:-
```

blogdb*"]

interval: 10s

timeout: 5s

retries: 5

logging:

driver: json-file

options:

max-size: "50m"

max-file: "3"

===== Redis缓存 =====

redis:

image: redis:7-alpine

container_name: blog-redis

restart: unless-stopped

```
command: redis-server /usr/local/etc/redis/redis.conf
```

ports:

- "6379:6379"

volumes:

- redis_data:/data

- `./redis/redis.conf:/usr/local/etc/redis/redis.conf:ro`

networks:

- blog-network

healthcheck:

```
test: ["CMD", "redis-cli", "ping"]
```

interval: 10s

timeout: 5s

retries: 5

logging:

driver: json-file

options:

max-size: "20m"

max-file: "3"


```
# ===== Certbot (SSL证书自动续期) =====
```


certbot:

image: certbot/certbot

container_name: blog-certbot

volumes:

- certbot-etc:/etc/letsencrypt

- certbot-var:/var/lib/letsencrypt

- `./frontend/dist:/var/www/html`

```
entrypoint: "/bin/sh -c 'trap exit TERM; while :; do certbot renew; sleep 12h & wa
```



```
it ${!*}; done;''
```

depends_on:

- nginx

===== 数据卷定义 =====

volumes:

postgres_data:

driver: local

redis_data:

driver: local

backend-data:

driver: local

certbot-etc:

driver: local

certbot-var:

driver: local

===== 网络定义 =====

networks:

blog-network:

driver: bridge

ipam:

config:

- subnet: 172.28.0.0/16

□1.3 各服务Dockerfile

后端Dockerfile:

```
# ===== 多阶段构建 =====

# 阶段1: 构建阶段
FROM python:3.11-slim AS builder

WORKDIR /build

# 安装构建依赖
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc libpq-dev && \
    rm -rf /var/lib/apt/lists/*

# 复制依赖文件并安装
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# 阶段2: 生产镜像
FROM python:3.11-slim AS production

# 创建非root用户
RUN groupadd -r appgroup && useradd -r -g appgroup appuser

# 安装运行时依赖
RUN apt-get update && \
    apt-get install -y --no-install-recommends libpq5 curl && \
    rm -rf /var/lib/apt/lists/* && \
    apt-get clean

# 从构建阶段复制已安装的包
COPY --from=builder /install /usr/local

# 设置工作目录
WORKDIR /app

# 复制应用代码
COPY . .

# 切换到非root用户
USER appuser

# 暴露端口
EXPOSE 8000

# 健康检查
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
    CMD curl -f http://localhost:8000/health || exit 1

# 启动命令
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--workers", "2"]
```

前端Dockerfile:

```
# 构建阶段：使用Node.js构建Vue项目
FROM node:18-alpine AS builder

WORKDIR /app

# 先复制package文件以利用Docker缓存
COPY package.json pnpm-lock.yaml ./

# 安装pnpm和依赖
RUN npm install -g pnpm && pnpm install --frozen-lockfile

# 复制源码并构建
COPY . .
RUN pnpm build

# 生产阶段：使用Nginx提供静态文件服务
FROM nginx:alpine AS production

# 复制自定义Nginx配置
COPY nginx.conf /etc/nginx/nginx.conf

# 从构建阶段复制产物
COPY --from=builder /app/dist /usr/share/nginx/html

# 暴露端口
EXPOSE 80

# 启动Nginx
CMD ["nginx", "-g", "daemon off;"]
```

前端Nginx配置:

```

server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # Gzip压缩
    gzip on;
    gzip_vary on;
    gzip_min_length 1024;
    gzip_types text/plain text/css text/xml text/javascript application/x-javascript application/xml+rss application/json;

    # SPA路由支持 (vue-router history模式)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # 静态资源缓存
    location ** \.(js|css|png|jpg|jpeg|gif|ico|svg|woff|woff2|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # API代理转发到后端
    location /api {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # 健康检查端点
    location /health {
        access_log off;
        return 200 'OK';
        add_header Content-Type text/plain;
    }
}

```

❑ 2. Nginx 反向代理配置 (1小时)

❑ 2.1 主Nginx配置

```
# nginx/nginx.conf

# 全局配置
worker_processes auto;
error_log /var/log/nginx/error.log warn;
pid /var/run/nginx.pid;

events {
    worker_connections 2048;
    use epoll;
    multi_accept on;
    *
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    # 日志格式
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for" '
        'rt=$request_time';

    access_log /var/log/nginx/access.log main;

    # 性能优化
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;
    keepalive_timeout 65;
    types_hash_max_size 2048;

    # Gzip全局设置
    gzip on;
    gzip_vary on;
    gzip_proxied any;
    gzip_comp_level 6;
    gzip_min_length 256;
    gzip_types
        text/plain
        text/css
        text/xml
        text/javascript
        application/json
        application/javascript
        application/xml+rss
        application/vnd.ms-fontobject
        font/opentype
        image/svg+xml;
```


安全头

```
add_header X-Frame-Options "SAMEORIGIN" always;
```



```
add_header X-Content-Type-Options "nosniff" always;
```

```
add_header X-XSS-Protection "1; mode=block" always;
```

```
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```


速率限制

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

```
limit_conn_zone $binary_remote_addr zone=conn_limit:10m;
```


上游服务器定义（后端API）

```
upstream backend_server {
```

server backend:8000;

```
keepalive 32; # 保持长连接数
```


加载站点配置

```
include /etc/nginx/conf.d/*.conf;
```


□2.2 站点配置文件

```

# nginx/conf.d/backend.conf

# HTTP -> HTTPS 重定向
server {
    listen 80;
    server_name api.yourdomain.com;
    return 301 https://$host$request_uri;
}

# HTTPS - API服务
server {
    listen 443 ssl http2;
    server_name api.yourdomain.com;

    # SSL证书配置
    ssl_certificate /etc/letsencrypt/live/api.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.yourdomain.com/privkey.pem;

    # SSL优化
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:50m;
    ssl_session_tickets off;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;
    ssl_prefer_server_ciphers off;

    # HSTS
    add_header Strict-Transport-Security "max-age=63072000" always;

    # 访问日志
    access_log /var/log/nginx/api_access.log main;
    error_log /var/log/nginx/api_error.log;

    # 限流
    limit_req zone=api_limit burst=20 nodelay;
    limit_conn conn_limit 10;

    # API代理
    location / {
        limit_except POST {
            deny all;
        }

        proxy_pass http://backend_server;

        # 请求头设置
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

```
proxy_set_header X-Forwarded-Proto $scheme;
```

```
proxy_set_header Connection "";
```


超时设置

```
proxy_connect_timeout 60s;
```



```
proxy_send_timeout 60s;
```

```
proxy_read_timeout 120s;
```


缓冲区设置

proxy_buffering on;

```
proxy_buffer_size 4k;
```

```
proxy_buffers 8 4k;
```

```
proxy_busy_buffers_size 8k;
```


Swagger文档（仅允许内网访问）

```
location /docs {
```

```
allow 127.0.0.1;
```

```
allow 10.0.0.0/8;
```

```
allow 172.16.0.0/12;
```

```
allow 192.168.0.0/16;
```


deny all;


```
proxy_pass http://backend_server;
```



```
# nginx/conf.d/frontend.conf

# HTTP -> HTTPS 重定向
server {
    listen 80;
    server_name www.yourdomain.com yourdomain.com;
    return 301 https://$host$request_uri;
}

# HTTPS - 前端静态网站
server {
    listen 443 ssl http2;
    server_name www.yourdomain.com yourdomain.com;

    # SSL证书
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;

    # SSL配置 (同上)
    ssl_session_timeout 1d;
    ssl_session_cache shared:SSL:50m;
    ssl_protocols TLSv1.2 TLSv1.3;

    # 根目录
    root /usr/share/nginx/html/frontend;
    index index.html;

    # 安全头
    add_header Content-Security-Policy "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; style-src 'self' 'unsafe-inline'; img-src 'self' data: https:;";
    always;
    add_header X-Frame-Options "SAMEORIGIN" always;

    # 静态资源长期缓存
    location ** \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2?|ttf|eot)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
        access_log off;
    }

    # SPA路由支持
    location / {
        try_files $uri $uri/ /index.html;
    }

    # 健康检查
    location /health {
        access_log off;
        return 200 'OK';
    }
}
```

```
add_header Content-Type text/plain;
```

3. CI/CD 流水线搭建 (1小时)

3.1 GitHub Actions 工作流

```
# .github/workflows/deploy.yml

name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${GITHUB_REPOSITORY}

jobs:
  # ===== 测试阶段 =====
  test:
    name: Run Tests
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_USER: testuser
          POSTGRES_PASSWORD: testpass
          POSTGRES_DB: testdb
        ports:
          - 5432:5432
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

      redis:
        image: redis:7
        ports:
          - 6379:6379
        options: >-
          --health-cmd "redis-cli ping"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
```

uses: actions/setup-python@v5

with:

python-version: '3.11'

cache: 'pip'

- name: Install dependencies

run: |

```
cd backend
```

```
pip install -r requirements.txt
```

```
pip install pytest pytest-cov pytest-asyncio httpx
```


- name: Run tests with coverage

run: |


```
cd backend
```

```
pytest tests/ -v --cov=app --cov-report=xml --cov-report=html
```


- name: Upload coverage report

```
if: success()
```

uses: actions/upload-artifact@v4

with:

name: coverage-report

path: backend/htmlcov/

===== 构建阶段 =====

build:

name: Build & Push Images

runs-on: ubuntu-latest

needs: test

```
if: github.event_name == 'push'
```


permissions:

contents: read

packages: write

steps:

- uses: actions/checkout@v4

- name: Set up Docker Buildx

uses: docker/setup-buildx-action@v3

- name: Log in to Container Registry

uses: docker/login-action@v3

with:

```
registry: ${env.REGISTRY **
```

username: \${github.actor} **


```
password: ${ secrets.GITHUB_TOKEN }
```


- name: Extract metadata for Docker

id: meta

uses: docker/metadata-action@v5

with:

```
images: ${ env.REGISTRY }/**/${ env.IMAGE_NAME }
```

tags: |

type=ref,event=branch

type=sha,prefix=

type=raw,value=latest,enable={{is_default_branch**

- name: Build and push Backend image

uses: docker/build-push-action@v5

with:

context: ./backend

push: true

```
tags: ${ steps.meta.outputs.tags **
```

```
labels: ${ steps.meta.outputs.labels }
```

cache-from: type=gha

cache-to: type=gha,mode=max

- name: Build and push Frontend image

uses: docker/build-push-action@v5

with:

context: ./frontend

push: true

```
tags: ${ env.REGISTRY } **/${ env.IMAGE_NAME } *-frontend:${ env.github.sha } **
```

```
labels: ${{ steps.meta.outputs.labels **
```

cache-from: type=gha

cache-to: type=gha,mode=max

===== 部署到生产环境 =====

deploy-production:

name: Deploy to Production

runs-on: ubuntu-latest

needs: build

```
if: github.ref == 'refs/heads/main'
```


environment:

name: production

url: <https://yourdomain.com>

steps:

- uses: actions/checkout@v4

- name: Deploy to server via SSH

uses: appleboy/ssh-action@v1.0.3

with:

```
host: ${ secrets.PROD_HOST **
```

```
username: ${ secrets.PROD_USER **
```

```
key: ${{ secrets.SSH_PRIVATE_KEY }}
```

script: |

```
cd /opt/blog-system
```


拉取最新代码


```
git pull origin main
```



```
# 更新环境变量（如果.env有变化）
```


使用新的镜像重新创建服务

```
docker compose -f docker-compose.prod.yml pull
```

```
docker compose -f docker-compose.prod.yml up -d --remove-orphans
```


清理旧镜像

```
docker image prune -af --filter "until=24h"
```


运行数据库迁移

```
docker compose -f docker-compose.prod.yml exec -T backend alembic upgrade
```

head


```
echo "' Deployment completed at $(date)'"
```


- name: Health check

run: |

```
sleep 30 # 等待服务启动
```


检查前端

```
curl -f https://yourdomain.com/health || exit 1
```


检查后端API

```
curl -f https://api.yourdomain.com/health || exit 1
```



```
echo "' All services are healthy"
```


- name: Notify deployment success

```
if: success()
```

uses: 8398a7/action-slack@v3

with:

status: success

channel: '#deployments'

webhook_url: \${ secrets.SLACK_WEBHOOK **

fields: repo,message,commit,author,action,eventName,ref,workflow

- name: Notify deployment failure

```
if: failure()
```


uses: 8398a7/action-slack@v3

with:

status: failure

channel: '#deployments'

webhook_url: \${ secrets.SLACK_WEBHOOK **

□3.2 生产环境配置

```
# docker-compose.prod.yml

version: '3.8'

services:
  backend:
    image: ghcr.io/yourusername/blog-system:main
    restart: always
    environment:
      - DATABASE_URL=postgresql://${POSTGRES_USER*}:${POSTGRES_PASSWORD*}@db:5432/${POSTGRES_DB*}
      - REDIS_URL=redis://redis:6379/0
      - SECRET_KEY=${SECRET_KEY*}
      - ENVIRONMENT=production
      - WORKERS=4
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2G
        reservations:
          cpus: '0.5'
          memory: 512M
    logging:
      driver: json-file
      options:
        max-size: "100m"
        max-file: "5"

  frontend:
    image: ghcr.io/yourusername/blog-system-frontend:main
    restart: always
    deploy:
      resources:
        limits:
          cpus: '0.5'
          memory: 256M

  db:
    image: postgres:15-alpine
    restart: always
    environment:
      POSTGRES_USER: ${POSTGRES_USER*}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD*}
      POSTGRES_DB: ${POSTGRES_DB*}
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./backups:/backups
    deploy:
```

resources:

limits:

cpus: '2.0'

memory: 4G

redis:

image: redis:7-alpine

restart: always

```
command: redis-server --maxmemory 512mb --maxmemory-policy allkeys-lru --appendonly
```


y yes

volumes:

- redis_data:/data

nginx:

image: nginx:alpine

restart: always

ports:

- "80:80"

- "443:443"

volumes:

- `./nginx/nginx.conf:/etc/nginx/nginx.conf:ro`

- `./nginx/conf.d:/etc/nginx/conf.d:ro`

- ./certs:/etc/nginx/certs:ro

- ./logs:/var/log/nginx

depends_on:

- backend

- frontend

□4. 监控与日志系统（1小时）

□4.1 应用健康检查端点

```

# health.py - FastAPI健康检查

from fastapi import APIRouter, Depends
from sqlalchemy import text
from datetime import datetime
import redis
import psutil
import platform

router = APIRouter(tags=["Health"])

def get_db_health(db):
    """检查数据库连接"""
    try:
        db.execute(text("SELECT 1"))
        return {"status": "healthy", "latency_ms": 0* # 实际应测量延迟
    except Exception as e:
        return {"status": "unhealthy", "error": str(e)*

def get_redis_health():
    """检查Redis连接"""
    try:
        r = redis.Redis(host='redis', port=6379)
        r.ping()
        return {"status": "healthy"*
    except Exception as e:
        return {"status": "unhealthy", "error": str(e)*

@router.get("/health")
async def health_check():
    """综合健康检查"""
    checks = {
        "database": get_db_health(get_db()),
        "cache": get_redis_health(),
        *

    overall_status = "healthy" if all(
        c["status"] == "healthy" for c in checks.values()
    ) else "unhealthy"

    return {
        "status": overall_status,
        "timestamp": datetime.utcnow().isoformat(),
        "version": "1.0.0",
        "checks": checks,
        "system": {
            "cpu_percent": psutil.cpu_percent(),
            "memory_percent": psutil.virtual_memory().percent,
            "disk_percent": psutil.disk_usage('/').percent,

```

```
"platform": platform.platform(),
```

```
"python_version": platform.python_version()
```



```
@router.get("/health/readiness")
```

```
async def readiness_check():
```

"""就绪检查（Kubernetes使用）"""

检查关键依赖是否可用

```
return {"status": "ready"*
```



```
@router.get("/health/liveness")
```

```
async def liveness_check():
```

"""存活检查（Kubernetes使用）"""

```
return {"status": "alive"}
```

□4.2 日志聚合方案

```
# logging_config.py - 结构化日志配置

import logging
import sys
import json
from pythonjsonlogger import jsonlogger

class CustomJsonFormatter(jsonlogger.JsonFormatter):
    """自定义JSON日志格式"""

    def add_fields(self, log_record, record, message_dict):
        super(log_record, record, message_dict)

        # 添加固定字段
        log_record['timestamp'] = self.formatTime(record, self.datefmt)
        log_record['level'] = record.levelname
        log_record['logger'] = record.name
        log_record['module'] = record.module
        log_record['function'] = record.funcName
        log_record['line'] = record.lineno

        # 添加上下文信息（如果存在）
        if hasattr(record, 'user_id'):
            log_record['user_id'] = record.user_id
        if hasattr(record, 'request_id'):
            log_record['request_id'] = record.request_id
        if hasattr(record, 'ip_address'):
            log_record['ip_address'] = record.ip_address

def setup_logging(service_name: str, level: str = "INFO"):
    """配置结构化日志"""

    # 根日志记录器
    root_logger = logging.getLogger()
    root_logger.setLevel(getattr(logging, level.upper()))

    # 控制台处理器（开发环境）
    console_handler = logging.StreamHandler(sys.stdout)
    console_handler.setFormatter(logging.Formatter(
        '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
    ))

    # JSON文件处理器（生产环境）
    file_handler = logging.FileHandler(f'/var/log/{service_name}.log')
    file_handler.setFormatter(CustomJsonFormatter())

    root_logger.addHandler(console_handler)
    root_logger.addHandler(file_handler)
```


设置第三方库的日志级别

```
logging.getLogger('uvicorn').setLevel(logging.INFO)
```

```
logging.getLogger('sqlalchemy.engine').setLevel(logging.WARNING)
```


在FastAPI中使用

```
# main.py
```

```
from logging_config import setup_logging
```



```
setup_logging('blog-api', 'INFO')
```



```
@app.middleware("http")
```

```
async def log_requests(request: Request, call_next):
```

"""请求日志中间件"""

```
request_id = str(uuid.uuid4())[:8]
```



```
logger.info(
```

```
f"Request started",
```

extra={

'request_id': request_id,

```
'method': request.method,
```

```
'url': str(request.url),
```

```
'ip_address': request.client.host
```



```
start_time = time.time()
```

```
response = await call_next(request)
```

```
process_time = time.time() - start_time
```



```
logger.info(
```

f"Request completed",

extra={

'request_id': request_id,

```
'status_code': response.status_code,
```

```
'process_time_ms': round(process_time * 1000, 2)
```



```
response.headers["X-Request-ID"] = request_id
```


return response

□ 4.3 Prometheus + Grafana 监控

```

# metrics.py - Prometheus指标导出

from prometheus_client import Counter, Histogram, Gauge, Info
from fastapi import Request
import time

# 定义指标
REQUEST_COUNT = Counter(
    'http_requests_total',
    'Total HTTP requests',
    ['method', 'endpoint', 'status']
)

REQUEST_LATENCY = Histogram(
    'http_request_duration_seconds',
    'HTTP request latency in seconds',
    ['method', 'endpoint'],
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0]
)

ACTIVE_USERS = Gauge(
    'active_users_current',
    'Currently active users'
)

DB_CONNECTION_POOL_SIZE = Gauge(
    'db_connection_pool_size',
    'Database connection pool size'
)

APP_INFO = Info(
    'application_info',
    'Application metadata'
)

# 初始化应用信息
APP_INFO.info({
    'version': '1.0.0',
    'name': 'blog-api'
})

class MetricsMiddleware:
    """Prometheus指标收集中间件"""

    async def __call__(self, request: Request, call_next):
        method = request.method
        endpoint = request.url.path

```

```
start_time = time.time()
```


try:

```
response = await call_next(request)
```

```
status = str(response.status_code)
```



```
except Exception as e:
```

```
status = '500'
```

raise

finally:

```
duration = time.time() - start_time
```



```
REQUEST_COUNT.labels(method=method, endpoint=endpoint, status=status).inc(
```



```
REQUEST_LATENCY.labels(method=method, endpoint=endpoint).observe(duration)
```


return response

在FastAPI中注册

```
app.add_middleware(MetricsMiddleware)
```


暴露metrics端点（供Prometheus抓取）

```
from prometheus_client import generate_latest
```



```
@app.get("/metrics")
```

```
async def metrics():
```

""Prometheus metrics endpoint""

```
return Response(generate_latest(), media_type="text/plain")
```

Grafana仪表盘JSON示例（简化版）：

```
{
  "dashboard": {
    "title": "Blog System Dashboard",
    "panels": [
      {
        "title": "Request Rate",
        "type": "graph",
        "targets": [
          {
            "expr": "sum(rate(http_requests_total[5m])) by (status)",
            "legendFormat": "{{status}}"
          }
        ]
      },
      {
        "title": "Response Latency (P99)",
        "type": "graph",
        "targets": [
          {
            "expr": "histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))",
            "legendFormat": "P99 Latency"
          }
        ]
      },
      {
        "title": "Error Rate",
        "type": "singlestat",
        "targets": [
          {
            "expr": "sum(rate(http_requests_total{status=~\"5..\"}[5m])) / sum(rate(http_requests_total[5m])) * 100"
          }
        ],
        "thresholds": [
          { "value": 1, "color": "green" },
          { "value": 5, "color": "yellow" },
          { "value": 10, "color": "red" }
        ]
      }
    ]
  }
}
```


*> 实践任务清单

☐任务1：容器化完整博客系统（2小时）

任务要求：

将Day2-4开发的博客前后端项目完全容器化：

- ☐ ✓ 为每个服务编写优化的Dockerfile
- ☐ ✓ 编写docker-compose.yml编排所有服务
- ☐ ✓ 配置Nginx反向代理
- ☐ ✓ 实现一键启动命令
- ☐ ✓ 添加健康检查
- ☐ ✓ 配置数据持久化
- ☐ ✓ 编写环境变量模板(.env.example)

验收步骤：

```
# 1. 克隆项目到服务器
git clone your-repo-url
cd blog-system

# 2. 配置环境变量
cp .env.example .env
vim .env # 编辑数据库密码、密钥等敏感信息

# 3. 一键启动所有服务
docker compose up -d --build

# 4. 检查服务状态
docker compose ps
# 应该看到所有服务状态为 Up

# 5. 查看日志
docker compose logs -f backend
docker compose logs -f nginx

# 6. 验证访问
curl http://localhost/health           # 前端
curl http://localhost/api/health       # 后端API
curl http://localhost:8000/docs        # Swagger文档

# 7. 数据库验证
docker compose exec db psql -U bloguser -d blogdb -c "\dt"

# 8. Redis验证
docker compose exec redis redis-cli ping
```

检验标准：

- ☐ [] docker compose up -d 能成功启动全部服务
- ☐ [] 所有容器状态为 healthy 或 up
- ☐ [] 前端页面能正常访问 (http://localhost)
- ☐ [] 后端API能正常工作 (http://localhost/api/docs)
- ☐ [] 数据库连接正常，表已自动创建
- ☐ [] Redis连接正常
- ☐ [] 重启后数据不丢失（数据持久化生效）

☐任务2：搭建自动化部署流水线（1.5小时）

任务要求：

在GitHub仓库中配置CI/CD：

- ☐ ✓ 推送代码自动触发测试
- ☐ ✓ 测试通过后自动构建Docker镜像
- ☐ ✓ 合并到main分支自动部署到服务器
- ☐ ✓ 部署成功/失败发送通知（Slack/邮件）
- ☐ ✓ 包含数据库迁移步骤

快速搭建指南：

```
# 1. 准备服务器环境
ssh user@your-server

# 安装Docker和Docker Compose
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# 安装Certbot (Let's Encrypt)
sudo apt install certbot

# 创建项目目录
sudo mkdir -p /opt/blog-system
sudo chown $USER:$USER /opt/blog-system
cd /opt/blog-system

# 克隆代码
git clone your-repo-url .

# 生成SSH密钥对 (用于GitHub Actions连接)
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ~/.ssh/github_deploy
cat ~/.ssh/github_deploy.pub # 将公钥添加到GitHub仓库的Deploy Keys

# 配置环境变量
cp .env.example .env
vim .env

# 首次手动部署
docker compose -f docker-compose.prod.yml up -d --build

# 2. 配置GitHub Secrets
# 进入仓库 Settings -> Secrets and variables -> Actions
# 添加以下secrets:
# - PROD_HOST: 服务器IP地址
# - PROD_USER: SSH用户名
# - SSH_PRIVATE_KEY: 私钥内容 (*.ssh/github_deploy的内容)
# - SLACK_WEBHOOK: Slack Webhook URL
# - POSTGRES_PASSWORD: 数据库密码
# - SECRET_KEY: JWT密钥

# 3. 推送代码触发CI/CD
git add .
git commit -m "feat: 添加CI/CD配置"
git push origin main

# 4. 观察Actions执行
# 进入仓库的Actions页面查看进度
```

检验标准：

- ☐ [] Push代码后Actions自动运行
- ☐ [] 测试阶段通过
- ☐ [] Docker镜像成功推送到Registry
- ☐ [] 服务器自动拉取新镜像并重启服务
- ☐ [] 部署成功收到Slack/邮件通知
- ☐ [] 网站更新为新版本且正常运行

** 推荐学习资源

☐ Docker相关

- ☐ Docker官方文档 - 最权威的学习资料
- ☐ Docker Compose文档 - 多容器编排
- ☐ 最佳实践 - 生产环境建议

☐ CI/CD相关

- ☐ GitHub Actions文档 - 完整指南
- ☐ Jenkins - 传统CI/CD工具
- ☐ GitLab CI/CD - GitLab内置CI/CD

☐ 监控相关

- ☐ Prometheus官方文档
- ☐ Grafana文档
- ☐ ELK Stack - 日志分析平台

☐ 工具推荐

- ☐ Portainer - Docker可视化界面
- ☐ Watchtower - 自动更新Docker镜像
- ☐ Fail2ban - 入侵防护
- ☐ UFW - 防火墙管理

✓ 今日自测题（精简版）

☐ 关键知识点检测

- ☐ Docker Compose中restart: unless-stopped 的含义？

- ☐ A. 不重启
- ☐ B. 除非手动停止，否则总是重启
- ☐ C. 只在出错时重启
- ☐ D. 永远不停止

- ☐ Nginx作为反向代理的优势？（选择所有适用）
- ☐ A. 负载均衡
- ☐ B. SSL终结
- ☐ C. 静态资源缓存
- ☐ D. 以上都是

- ☐ CI/CD中，什么时候应该运行集成测试？
- ☐ A. 只在本地开发时
- ☐ B. 每次代码提交时
- ☐ C. 仅在发布前
- ☐ D. 手动触发时

- ☐ 为什么生产环境不应该使用latest 标签？
- ☐ A. 最新版本可能有bug
- ☐ B. 无法确定正在运行的版本
- ☐ C. 回滚困难
- ☐ D. 以上都是

- ☐ 健康检查（Health Check）的主要目的？
- ☐ A. 监控CPU使用率
- ☐ B. 检测服务是否正常运行
- ☐ C. 收集日志
- ☐ D. 备份数据

☐答案

- ☐ B - 除非被手动停止或Docker本身停止，否则容器会自动重启。
- ☐ D - Nginx具备所有这些能力。
- ☐ B - 持续集成的核心是每次提交都运行测试。
- ☐ D - latest标签不可预测，不利于版本管理和回滚。
- ☐ B - 健康检查用于确认服务是否可用，便于自动恢复。

** 今日总结

☐关键收获

- ☐ _____
- ☐ _____
- ☐ _____

☐明日预告

Day 6: AI基础理论与国内大模型应用 将涵盖:

- ☐ 大语言模型原理 (Transformer架构)
- ☐ Prompt Engineering技巧
- ☐ DeepSeek API调用与实践
- ☐ 豆包 (火山引擎) API接入
- ☐ AI能力集成到现有项目

准备工作:

- ☐ [] 注册DeepSeek和火山引擎账号
- ☐ [] 申请API Key
- ☐ [] 了解基本的Prompt概念
- ☐ [] 准备一些文本处理需求场景

** 实战项目: 生产级部署方案 ☐

☐项目概览

项目名称: Deployment - CI/CD与容器化部署 路径:05-运维与部署实践/deployment/ 完成度: ✓
100% 文件数: 8个核心文件 技术栈: Docker Compose + GitHub Actions + Nginx + SSL/TLS

☐核心特性

✓ Docker多阶段构建 - 优化镜像体积, 生产环境安全加固 ✓ Docker Compose编排 - 全栈服务一键启动 (Web+DB+Cache+Nginx) ✓ GitHub Actions CI/CD - 自动化测试 构建 部署流水线 ✓ Nginx生产配置 - SSL/TLS + Gzip压缩 + 限流 + 静态资源缓存 ✓ 健康检查体系 - 应用级 + 数据库级 + 缓存级监控

☐项目架构

```

deployment/
+-- .github/workflows/
|   +-- ci-cd.yml          # 完整CI/CD流水线
|       +-- Lint & Test阶段
|       +-- Build & Push阶段
|       +-- Deploy & Notify阶段
|
+-- docker-compose.prod.yml # 生产环境编排
|   +-- 资源限制 (CPU/Memory)
|   +-- 健康检查配置
|   +-- 日志轮转策略
|   +-- 网络隔离设置
|
+-- nginx/
|   +-- prod.conf          # 生产Nginx主配置
|       +-- HTTP*HTTPS重定向
|       +-- SSL/TLS优化
|       +-- Gzip压缩
|       +-- 请求限流
|       +-- 安全头设置
|   +-- ssl/
|       +-- README.md      # SSL证书管理指南
|       +-- Let's Encrypt自动续期
|       +-- 自签名证书生成
|       +-- 证书安全最佳实践

```

□ CI/CD流水线详解

```

# 触发条件
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

# 流水线阶段
Jobs:
  1. test:          # 单元测试 + 覆盖率报告
  2. build:         # Docker镜像构建 + 推送Registry
  3. deploy-prod:  # SSH部署到服务器 + 健康检查
  4. notify:        # Slack/邮件通知部署结果

```

□ Nginx生产优化特性

特性	配置	效果
SSL/TLS	TLSv1.2+ / HSTS	数据传输加密
Gzip	text/css, application/json	带宽节省60-80%
限流	10r/s per IP	防DDoS攻击
静态缓存	1年（带hash文件名）	减少服务器负载
日志	JSON格式 + 轮转	便于ELK分析

快速启动

```
# 1. 进入项目目录
cd 05-运维与部署实践/deployment

# 2. 复制环境变量模板
cp ../03-后端开发技能提升/blog-api/.env.example .env
vim .env # 编辑配置（数据库密码、密钥等）

# 3. 构建并启动所有服务（开发环境）
docker compose up -d --build

# 4. 查看服务状态
docker compose ps
# 应该看到: backend, frontend, nginx, db, redis 全部 Up

# 5. 访问服务
# 前端: http://localhost
# API: http://localhost/api/docs (Swagger)
# Nginx健康检查: http://localhost/health

# 6. 查看日志
docker compose logs -f backend # 后端日志
docker compose logs -f nginx   # 访问日志
```

生产部署步骤

```
# 1. 使用生产配置启动
docker compose -f docker-compose.prod.yml up -d --build

# 2. 初始化数据库
docker compose exec backend alembic upgrade head

# 3. 配置SSL证书 (Let's Encrypt)
# 参照 nginx/ssl/README.md 操作指南

# 4. 设置GitHub Actions Secrets
# PROD_HOST, PROD_USER, SSH_PRIVATE_KEY, SLACK_WEBHOOK

# 5. 推送代码触发自动部署
git push origin main
```

☐ 验收标准

- ☐ [] docker compose up -d 成功启动全部服务
- ☐ [] 所有容器状态为 healthy
- ☐ [] HTTPS访问正常 (SSL证书有效)
- ☐ [] Gzip压缩生效 (响应头包含 Content-Encoding: gzip)
- ☐ [] API文档可访问 (<http://yourdomain.com/api/docs>)
- ☐ [] 日志正常输出且支持轮转
- ☐ [] GitHub Actions流水线成功执行

*☐ 模块导航

Day 4: 数据库技术应用 | Day 6: AI基础理论与国内大模型应用 | ☒ 返回课程首页

√* Day 5 完成! 你的应用已经具备生产级部署能力!

明日将进入激动人心的AI世界, 为你的应用注入智能!